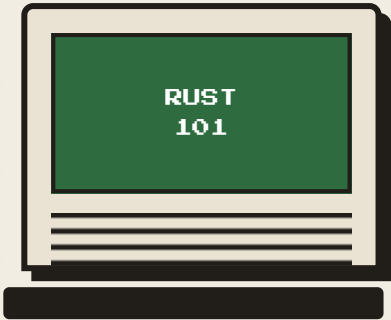




MINI CURSO DE RUST

PARA QUEM TESTA COM CYPRESS E PLAYWRIGHT

Você já sabe automatizar browsers. Agora vamos abrir o console e ler o **esquemático**: um curso de Rust do zero usando o código-fonte real do **SNES Xperience** como cartucho de estudos. Nada de exemplos com `foo` e `bar` — cada aula lê um arquivo de verdade do emulador.

NÍVEL

Iniciante total em Rust

PRÉ-REQUISITO

Cypress/Playwright básico

CONTEÚDO

11 aulas + projeto final

00 ANTES DO POWER

COMO USAR ESTE GUIA

Este curso foi escrito para **quem nunca escreveu Rust** — mas já passou horas esperando um `cypress run` terminar. Cada aula apresenta um conceito e mostra onde ele vive **no código real do emulador**, com o caminho do arquivo para você abrir no editor.

As aulas se apoiam nas anteriores: leia em ordem, na primeira passada. Trave na **aula 8** (posse e empréstimo)? Faz parte — é a chefe final de Rust e todo mundo trava nela.

BOX VERDE — ANALOGIA DE QA

Traduz o conceito de Rust para o seu dia a dia de automação: locators, fixtures, `cy.task`, CI, flaky tests. É a ponte entre os dois mundos.

BOX ROXO — DESAFIO

Exercício curto para fazer no próprio projeto (ou num cargo `new` descartável). Nenhum exige mais que 15 linhas. Gabarito comentado dentro do próprio box.

AULA	O QUE VOCÊ APRENDE	ONDE NO CONSOLE
1	O que é Rust e por que um emulador o usa	README.md
2	Cargo (o “npm de Rust”) e o mapa do projeto	Cargo.toml
3	Variáveis, tipos e funções	emulation/core.rs
4	Structs e enums — modelando o mundo	domain/rom.rs
5	<code>match</code> — o switch que não deixa caso faltar	domain/rom.rs
6	<code>Option</code> e <code>Result</code> — o fim do <code>null</code>	domain/rom.rs
7	Coleções e iteradores (<code>map/filter/collect</code>)	emulation/core.rs
8	Posse e empréstimo — a chefe final	emulation/core.rs
9	Módulos e crates — como o código se organiza	domain/lib.rs
10	Testes em Rust (sua casa, afinal)	domain/rom.rs
11	Chefes opcionais: <code>unsafe</code> , <code>threads</code> , <code>macros</code>	ntsc · app

MINDSET ANTES DE TUDO

Em Rust, o **compilador é o seu CI mais exigente**: roda antes de tudo, bloqueia o merge de qualquer bug de memória e devolve o erro **com a linha e a correção sugerida**. Ele é chato pelo mesmo motivo que um bom teste é chato: para nada quebrar em produção.

01 O QUE É RUST

E POR QUE O CONSOLE FOI CONSTRUÍDO COM ELE

Rust é uma linguagem **compilada**: você escreve o código, roda o compilador e ele gera um programa nativo — um .app/ .exe de verdade, sem interpretador viajando junto, como em C++. Nada de node embutido.

A promessa dele é ousada: **velocidade de C++ com um compilador que recusa programas com chance de erro de memória** antes de você apertar F5. Acessar memória já liberada, pisar em ponteiro solto — em Rust, em geral, isso vira erro de **compilação**, não bug de produção.

NO SEU MUNDO (JS/TS)	NO MUNDO RUST
node interpreta seu JS em runtime	rustc compila para código de máquina nativo
undefined is not a function estoura em produção	o equivalente estoura na compilação , com a linha certa
null e undefined escapam quando ninguém vê	não existem — existe Option, que o compilador te obriga a abrir
try/catch é opcional (e vive esquecido)	Result é contrato: a assinatura declara a falha
o teste E2E é a última linha de defesa	o compilador é a primeira — e nunca fica verde por engano

ANALOGIA DE QA

Você conhece a diferença entre o bug que o Cypress pegou no pipeline e o que o cliente pegou em produção? O compilador de Rust é o estágio de CI mais rigoroso possível: roda **antes de tudo**, não deixa o merge passar e explica o erro. É o TypeScript estrito elevado à enésima potência — só que caçando bugs de **memória**, não só de tipo.

Por que um emulador de SNES precisa disso?

60 fps

O loop roda 60 vezes por segundo. Código lento = jogo engasgado. Rust compila tão rápido quanto C++.

Mundo C

O “cérebro” dos jogos (o core snes9x) é uma biblioteca C carregada em runtime. Precisa falar C sem se machucar.

Ponteiros

Frames de vídeo, memória do cartucho, áudio — tudo é buffer. É exatamente onde C++ morde.

02

CARGO É O SEU NPM

CONHECENDO O TERRENO DO PROJETO

Cargo é a ferramenta oficial que acompanha todo projeto Rust: compila, testa, baixa bibliotecas e gera documentação. Se você sabe npm, já sabe Cargo — só muda o nome.

VOCÊ JÁ FAZ (NPM)	O CARGO CHAMA DE
package.json	Cargo.toml — mesmo papel: nome, versão, dependências
npm install	cargo build — baixa dependências e compila tudo
cypress run / npx jest	cargo test — roda TODOS os testes do workspace
npm start	cargo run — compila e executa o programa
package-lock.json	Cargo.lock — trava versões exatas para todo mundo
registry do npm	crates.io — o “npmjs” de Rust, mesmo esquema

Um pacote de Rust se chama **crate**. O SNES Xperience é um **workspace** — o monorepo de Rust — com seis crates, cada um com uma responsabilidade:

domain
O que é uma ROM, como identificá-la, catálogo, cheats. Regras do negócio.

emulation
Conversa com os cores libretto (snes9x) carregados em tempo de execução.

platform
Janela, teclado, mouse, áudio — e o desenho do console/TV.

ntsc
O filtro que faz a imagem parecer TV de tubo dos anos 90.

ra
RetroAchievements: os troféus dos jogos.

app
Cola tudo e gera os programas (xperience, selector...).

REPARE NO DETALHE

O crate **domain** nem sabe que janela existe. O comentário no topo do `crates/domain/src/lib.rs` diz, em inglês: *“Domain layer: ROM identity, catalogue and No-Intro naming. **No SDL.**”* (SDL é a biblioteca gráfica — ou seja, nada de tela aqui.)

Cada crate tem **uma** responsabilidade, e o que é útil para os outros é reexportado na vitrine — a aula 9 abre essa vitrine.

PRIMEIRO COMANDO (AQUECIMENTO)

Na raiz do projeto, rode `cargo test`. Deve aparecer uma lista de testes e um `test result: ok`. Você acabou de compilar e validar o emulador inteiro — e já viu o formato de saída do Cargo, que vai acompanhar você o curso todo.

03

VARIÁVEIS E FUNÇÕES

RUST COM PICKET FENCE — CÓDIGO REAL, DIRETO DO CONSOLE

Este método real vive em `crates/emulation/src/core.rs` e responde “quantos bytes tem um pixel” conforme o formato de vídeo que o core anunciou (leia por cima do `match` — aula 5):

crates/emulation/src/core.rs

RUST

```
impl PixelFormat {
    pub fn bytes_per_pixel(self) -> usize {
        match self {
            PixelFormat::Rgb1555 | PixelFormat::Rgb565 => 2,
            PixelFormat::Xrgb8888 => 4,
        }
    }
}
```

Tipos explícitos depois de `:` (parâmetros) e `->` (retorno); `usize` é o inteiro “tamanho natural da máquina”. O estilo é `snake_case` — e o compilador *cobra* se você fugir dele. Sem `pub` na frente, uma função é **privada** do arquivo. Agora variáveis, de `crates/domain/src/rom.rs`:

crates/domain/src/rom.rs

RUST

```
let header_len = if bytes.len() % 1024 == 512 { 512 } else { 0 };
let rom = &bytes[header_len..]; // fatia: os bytes a partir do header
```

ANALOGIA DE QA

Se em JS você já aderiu ao “`const` por padrão, `let` só quando precisa” — Rust concorda **tão** forte que inverte: `let` é imutável como `const`, e mutabilidade se pede no balcão: `let mut total = 0;`

DOIS DETALHES QUE CONFUNDEM

1. O `if` acima **produz um valor** (vai para `header_len`) — em Rust, quase tudo é expressão; por isso não existe operador ternário.
2. A última expressão de uma função, **sem ponto-e-vírgula**, é o retorno. `count()` sem `;` já é o `return`.

DESAFIO 3 — FORMATADOR DE TAMANHO

Dentro do `impl RomId` em `crates/domain/src/rom.rs` adicione e depois rode `cargo check -p xperience-domain`:

```
pub fn human_size(&self) -> String {
    if self.rom_len >= 1024 * 1024 {
        format!("{:.1} MiB", self.rom_len as f64 / 1048576.0)
    } else {
        format!("{:.1} KiB", self.rom_len as f64 / 1024.0)
    }
}
```

`format!` monta texto (`{:.1}` = uma casa decimal), `as f64` converte números e `&self` é “o próprio objeto”.

04

STRUCTS E ENUMS

MODELANDO O MUNDO DO CONSOLE

O tipo que representa “a identidade de uma ROM” — o que a estante guarda sobre cada cartucho — é uma **struct** em `crates/domain/src/rom.rs`:

```
crates/domain/src/rom.rs RUST  
  
#[derive(Debug, Clone)]           // código gerado de graça (abaixo)  
pub struct RomId {  
    pub header_len: usize,         // header antigo pulado (0 ou 512)  
    pub crc32: String,             // hash que identifica a ROM  
    pub internal_name: Option<String>, // título interno, se houver  
    pub mapper: Mapper,           // chip de mapeamento do cartucho  
}
```

Cada campo tem nome, tipo e o seu próprio pub. Os comentários `///` viram **documentação** (o cargo `doc` gera páginas HTML com eles). E o atributo `#[derive(...)]` pede ao compilador para gerar código entediante: `Debug` = imprimir no terminal, `Clone` = copiar, `PartialEq` = comparar com `==`. Detalhe elegante: se a variável tem o mesmo nome do campo (`crc32`), não precisa escrever `crc32: crc32`.

E o **enum** — o tipo do chip de mapeamento e do controle:

```
rom.rs + emulation/core.rs RUST  
  
pub enum Mapper { LoRom, HiRom, Unknown }  
  
pub enum Button { B, Y, Select, Start, Up, Down,  
                 Left, Right, A, X, L, R } // o pad de SNES inteiro
```

Um valor `Mapper` é **exatamente uma** das três variantes — nunca duas, nunca nenhuma, nunca um valor inventado. Não existe `mapper = 17` torcendo para ninguém notar: o compilador não deixa. Enums também têm métodos (a `bytes_per_pixel` da aula 3 era um método do enum `PixelFormat`).

ANALOGIA DE QA

Em TypeScript você modelaria `type Mapper = 'LoRom' | 'HiRom' | 'Unknown'`. O enum de Rust é essa união com superpoderes: cada variante pode **carregar dados próprios** (você verá com os erros, na aula 6) e o compilador conhece a lista completa de casos. Adicione um caso novo e ele aponta **todo lugar que precisa se adaptar** — como trocar um contrato de fixture e ver o schema apontar cada teste afetado.

DESAFIO 4 — O CHIP SUPER FX

Adicione a variante `SuperFx` ao enum `Mapper` (era o chip coprocessador de jogos como *Star Fox*). Rode `cargo check` no workspace inteiro. Se algum `match` não cobrir o caso novo, o compilador aponta arquivo e linha exatos. Esse é o momento “uau”: mude o mundo e deixe o compilador te dar a lista de tarefas.

05

MATCH

O INTERRUPTOR DE TREM: EXAUSTIVO POR LEI

match pega um valor e escolhe um ramo conforme ele é — o switch, mas **exaustivo**: esqueceu um caso, não compila. Este é real — como o emulador decide se a ROM é LoROM ou HiROM:

```
crates/domain/src/rom.rs · read_internal_header RUST

let lo = title_at(rom, LO); // título no endereço do LoROM
let hi = title_at(rom, HI); // título no endereço do HiROM
match (score(&lo), score(&hi)) {
    (l, h) if h > l => (Mapper::HiRom, hi), // o hi parece texto?
    (l, _) if l > 0 => (Mapper::LoRom, lo),
    _                => (Mapper::Unknown, None), // ninguém parece
}
```

Três novidades: match (a, b) casa uma **tupla** (par de valores); (l, h) if h > l é um padrão com **guarda** (uma condição extra); e _ é o “qualquer outra coisa”. Padrões também **desempacam** dados de dentro — o Some(s) abaixo extrai o valor e batiza-o de s:

```
crates/domain/src/rom.rs · score RUST

fn score(name: &Option<String>) -> usize {
    match name {
        None => 0,
        Some(s) => s.chars().filter(|c| c.is_ascii_alphanumeric()).count(),
    }
}
```

ANALOGIA DE QA

É o **router de comandos** do seu app de testes: cada comando tem um handler. A diferença é que aqui, quando amanhã surgir um comando novo e você esquecer dele, **o projeto nem compila**. O maior match do emulador é assim: dezenas de comandos que o core snes9x manda (emulation/core.rs, função environment_cb), com um _ => false final de “comando desconhecido, não sei responder”.

DESAFIO 5 — NTSC RODA A 60, PAL RODA A 50

```
enum Regiao { Ntsc, Pal }

fn fps(regiao: Regiao) -> f64 {
    match regiao {
        Regiao::Ntsc => 60.0,
        Regiao::Pal  => 50.0, // Europa: 5/6 da velocidade
    }
}
```

Agora quebre de propósito: comente o caso Pal e tente compilar — a mensagem diz non-exhaustive patterns e mostra o caso faltando. É essa proteção que você vai sentir falta nas outras linguagens.

06 OPTION E RESULT

O FIM DO NULL E DAS EXCEÇÕES SURPRESA

Rust **não tem null e não tem exceções**. No lugar, dois enums padrão que carregam dados (repare nas variantes com conteúdo):

definições da biblioteca padrão

RUST

```
enum Option<T> { Some(T), None } // "pode não existir"
enum Result<T, E> { Ok(T), Err(E) } // "pode falhar"
```

Option para “pode não existir” (o título interno da ROM — nem toda tem um válido). Result para “pode falhar” (ler um arquivo). A assinatura da função **declara** a falha e **qual** o tipo do erro:

crates/domain/src/rom.rs

RUST

```
pub enum RomError {
    #[error("could not read ROM {path}: {source}")]
    Read { path: String, source: std::io::Error },
    #[error("file is too small to be a SNES ROM ({0} bytes)")]
    TooSmall(usize),
}

pub fn from_bytes(bytes: &[u8]) -> Result<Self, RomError> {
    if bytes.len() < 0x8000 {
        return Err(RomError::TooSmall(bytes.len()));
    }
    // ... calcula hashes e devolve Ok(RomId { ... })
}
```

Quem chama **não consegue ignorar** a possibilidade de erro: para chegar ao RomId, é obrigatório passar pelo Result. E o operador ? é o atalho que você mais usará — de RomId::from_path:

crates/domain/src/rom.rs · o operador ?

RUST

```
let bytes = fs::read(path).map_err(|source| RomError::Read { ... })?;
Self::from_bytes(&bytes) // só chega aqui se o read deu certo
```

O ? quer dizer: “se deu errado, devolva o erro agora; se deu certo, me dê o valor e siga”. O caminho feliz se escreve como caminho feliz — sem pirâmide de try/catch. Helpers úteis: unwrap_or(padrao) (valor alternativo), and_then(...) (encadear buscas) e take() (pegar o valor deixando vazio — o frame de vídeo usa).

ANALOGIA DE QA

Pense em toda função perigosa devolvendo { ok, valor } — e a linguagem te **obrigando** a abrir esse objeto antes de usar o valor. O try/catch de JS é opcional e vive esquecido; o Result de Rust é contratual. O Cypress falha o teste no primeiro erro que aparece; Rust falha a **compilação** no primeiro erro não tratado. Mesma filosofia, estágio mais cedo.

07 ITERADORES

SUA ESTEIRA DE MAP/FILTER, AGORA COM FISCAL

A coleção cavalo de batalha é o `Vec<T>` (lista que cresce): `Vec<u8>` guarda os pixels de um frame, o conteúdo de uma ROM. O array de tamanho fixo também existe — `[Button; 12]`, os doze botões do pad. E o processamento é uma **esteira**:

```
crates/domain/src/rom.rs · score, por dentro RUST

s.chars()                // 1. esteira de caracteres
.filter(|c| c.is_ascii_alphanumeric()) // 2. só letras e números
.count()                 // 3. consome e conta

// o mesmo esquema encaixa etapas até virar lista de novo:
let valid_extensions = cstr.split('|') // "sfc|smc|swc" → 3 nomes
    .filter(|s| !s.is_empty())
    .map(|s| s.to_ascii_lowercase()) // tudo minúsculo
    .collect::<Vec<String>>();        // esteira → Vec
```

As `|c| ...` são **closures**: funções anônimas — suas conhecidas arrow functions de JS. O modelo mental: a esteira só roda quando alguém consome (`count`, `collect`, `for`); até lá, é preguiçosa e não aloca nada. O `filter` acima não criou lista intermediária nenhuma.

EM JS VOCÊ ESCRIVE	EM RUST, O EQUIVALENTE
<code>arr.map(x => ...)</code>	<code>iter().map(x ...)</code>
<code>arr.filter(x => ...)</code>	<code>iter().filter(x ...)</code>
<code>arr.find(x => ...)</code>	<code>iter().find(x ...)</code> — devolve <code>Option</code> !
<code>arr.reduce((a,b) => ...)</code>	<code>iter().fold(inicial, a,b ...)</code>
<code>[...arr].sort()</code> (copia antes)	<code>iter().sorted().collect::<Vec<_>>()</code>

ANALOGIA DE QA

É o `Array.prototype` de JS — só que com o compilador conferindo cada etapa e **sem alocação escondida**. E repare no `find`: em Rust ele devolve um `Option`, porque “não achei” é um resultado possível — nada de `undefined` escapando pela tubulação.

DESAFIO 7 — A ESTEIRA DO CATALOGO

Dado `let jogos = vec!["Zelda", "mario world", "F-ZERO"];`, escreva uma única expressão que devolva um `Vec<String>` com os nomes em maiúsculas (`to_uppercase()`) dos jogos com **mais de 5 caracteres** (`chars().count()`). Esperado: `["MARIO WORLD", "F-ZERO"]` — “Zelda” fica de fora: tem exatamente 5. Depois troque para `>= 5` e veja mudar só na borda.

OS

POSSE E EMPRÉSTIMO

A CHEFE FINAL — VÁ COM CALMA, TODO MUNDO TRAVA AQUI

A REGRA DE OURO

Todo valor tem exatamente um dono — e quando o dono sai de cena, o valor é destruído. Não existe free nem coletor de lixo: a memória é liberada no fim do escopo, automaticamente. Quem precisa usar o valor sem ser o dono, **empresta**:

&T — empréstimo **somente leitura**. Pode ter quantos quiser ao mesmo tempo. · **&mut T** — leitura e escrita. **Só um** por vez, e nenhum **&** junto.

Por que essas duas regras bastam? Porque praticamente todo bug de memória em C/C++ é alguém escrevendo num dado enquanto outro lê. Com as regras acima, isso **não compila**. Veja no emulador — o frame de vídeo é o exemplo perfeito:

```
emulation/core.rs                                     RUST

/// Um quadro de vídeo do core.
/// "Owns its pixels."
pub struct Frame {
    pub width: u32,
    pub height: u32,
    pub pixels: Vec<u8>, // o frame É O DONO
}

impl Drop for Core {
    fn drop(&mut self) { /* desliga o core C */ }
}
```

O Core entrega um Frame para quem pede (`take_frame`) — e o frame leva **os pixels dele**. Quando o frame morrer, os pixels morrem com ele. Zero vazamento, garantido pelo compilador.

E o `impl Drop` é o destrutor: quando o Core sai de cena, ele avisa o core C sozinho. Ninguém chama `retro_deinit` “à mão” no app — teardown automático, como uma boa fixture `afterEach`, só que garantida pela linguagem.

Nas assinaturas do Core, os termos do empréstimo são declarados:

```
crates/emulation/src/core.rs                         RUST

pub fn system_name(&self) -> &str // "me empresta o nome, só leitura"
pub fn audio(&self) -> &[i16]     // "me empresta o áudio"
pub fn run(&mut self)             // "preciso me MODIFICAR para rodar"
```

O compilador garante que ninguém guarda o `&[i16]` do áudio enquanto chama `run()` — que limpa e recheia o áudio. O bug de ponteiro pendente que viraria flaky test às 3h da manhã: **impossível por construção**.

ANALOGIA DE QA

É o modelo de **browser context do Playwright** levado ao extremo: cada valor tem um dono responsável pelo ciclo de vida (fixture com teardown automático). Vários podem **ler** ao mesmo tempo (`&`); se alguém vai **escrever** (`&mut`), é exclusivo — sem leitor junto. A data race que você caçaria com `retry` e café não compila.

SE O COMPILADOR TE XINGAR

Ele é chato como CI vermelho — e útil pelo mesmo motivo: as mensagens apontam a linha, explicam a regra e sugerem a correção.

09

MÓDULOS E CRATES

A VITRINE DO CRATE — EXPORTS DE VERDADE, NÃO CONVENÇÃO

Dentro de um crate, o código se divide em **módulos** (arquivos). A “vitrine” de cada crate é o `lib.rs` — ele decide o que a rua vê. Este é o do `domain`, quase inteiro:

```
crates/domain/src/lib.rs RUST

//! Domain layer: ROM identity, catalogue and No-Intro naming. No SDL.

pub mod rom;           // existe o módulo rom — código em src/rom.rs
pub mod library;

pub use rom::{Mapper, RomError, RomId}; // re-exporta: vitrine
pub use library::{scan, ScannedRom, ROM_EXTS};
```

`pub mod rom;` declara o módulo (o arquivo `rom.rs` que estudamos). `pub use rom::RomId;` **reexporta** — é por isso que os outros crates escrevem `use xperience_domain::RomId;` e não `...::rom::RomId;`. O `//!` no topo é documentação do módulo. Sem `pub`? Privado — invisível para fora.

Entre crates, a dependência é declarada no `Cargo.toml` do consumidor — `app` declara `domain`, `emulation`, `platform...` por **path**, igual aos `npm workspaces` que você já viu em monorepos JS. E as de terceiros (`serde`, `anyhow`) são versionadas uma vez só no `Cargo.toml` raiz, para os seis crates usarem a mesma versão.

O uso, no código do emulador:

```
crates/app/src/runner.rs · linha 1089 RUST

match xperience_domain::RomId::from_bytes(&rom_bytes) { /* ... */ }
```

ANALOGIA DE QA

É o `index.ts` de barrel + `exports` do `package.json` — só que com privacidade **fiscalizada**: se você importar algo que a vitrine não exportou, nem compila. Nada de alcançar arquivo interno por caminho relativo e torcer.

DESAFIO 9 — QUEM SABE O SHA-1?

Quando o `app` chama `RomId::from_bytes(...)`, qual dos dois crates “sabe” calcular SHA-1: o `app` (que **usa**) ou o `domain` (que **executa**)? Confira: procure `sha1` nos dois `Cargo.toml`. Essa separação — quem usa a informação não é quem sabe calcular — é o motivo do projeto estar fatiado assim.

10

TESTES

SUA CASA, AFINAL — UNIT TESTS EM MILISSEGUNDOS

Testes moram **no mesmo arquivo** do código, num módulo que **só existe quando se roda testes** (`#[cfg(test)]`) — zero custo no programa final. O coração da identificação de ROMs:

```
crates/domain/src/rom.rs · fim do arquivo RUST

#[cfg(test)]
mod tests {
    use super::*; // importa tudo do arquivo de cima

    #[test]
    fn strips_copier_header() {
        let mut rom = vec![0u8; 0x8000]; // ROM "nova" de 32 KiB
        // ... grava um título no endereço 0x7FC0 ...
        let bare = RomId::from_bytes(&rom).unwrap();
        let mut headered = vec![0u8; 512]; // + 512 bytes de lixo
        headered.extend_from_slice(&rom);
        let hd = RomId::from_bytes(&headered).unwrap();
        assert_eq!(hd.sha1, bare.sha1); // hashes IGUAIS
    }
}
```

A HISTÓRIA QUE O TESTE CONTA

Cartuchos copiados nos anos 90 vinham com **512 bytes de lixo** (header de copiadora) no começo. O teste garante que, com header ou sem, o hash é o **mesmo** — é isso que faz a sua ROM bater com o banco No-Intro e com o RetroAchievements.

Rodar: `cargo test` (tudo) ou `cargo test -p xperience-domain strips` (um crate, filtrando por nome). O `assert_eq!` falha mostrando left/right com diff — seu `expect().to.equal`.

ANALOGIA DE QA

São **unit tests** em milissegundos — sem browser, sem wait, sem flaky. O “E2E” aqui é abrir o emulador e jogar; o `cargo test` é a base barata que roda em todo push.

DESAFIO 10 — TESTE O SEU `human_size`

Da aula 3. Dentro do módulo tests de `crates/domain/src/rom.rs`:

```
#[test]
fn human_size_formats() {
    let rom = vec![0u8; 0x8000]; // 32 KiB de zeros
    let id = RomId::from_bytes(&rom).unwrap();
    assert_eq!(id.human_size(), "32.0 KiB");
}
```

Rode `cargo test -p xperience-domain` — se der fail, leia o diff de left/right. TDD completo no emulador. (E sim, `.unwrap()` em teste é aceitável: ali você *quer* que a falha estale alto.)

11

CHEFES OPCIONAIS

UNSAFE, THREADS E MACROS — RECONHECER JÁ BASTA (POR ENQUANTO)

Você já tem o essencial. Estas são as regiões avançadas do código — a meta aqui é só **saber que existem** e o que significam quando aparecerem no caminho.

CHEFE 1 — FFI E UNSAFE: A FRONTEIRA COM O C

O filtro de TV conversa com uma biblioteca escrita em C — e o crate emulation carrega o snes9x (.dylib/.dll) em runtime:

```
crates/ntsc/src/lib.rs RUST

extern "C" {
    // funções escritas em C
    fn snes_ntsc_blit(/* ... */); // o efeito TV de tubo
}
```

unsafe é o bloco que diz: *“a partir daqui, eu assumo a responsabilidade que o compilador não consegue checar”*. O projeto os cerca: blocos pequenos, comentados com o porquê (Safety: ...), escondidos atrás de funções seguras.

ANALOGIA DE QA

É o `cy.task`: sai do sandbox do browser para rodar no Node — poder real, com fronteira documentada. O resto do código de teste continua seguro; o risco fica isolado num lugar só, explicado.

CHEFE 2 — THREADS COM CANAIS

A checagem de update não pode travar o app esperando a internet. Roda numa thread de fundo e responde por um **canal**:

```
std::thread::spawn(move || {
    let aviso = check();
    sender.send(aviso).ok();
});
```

Os dados **mudam de dono** ao cruzar a thread (aula 8!). A data race simplesmente não compila.

CHEFE 3 — MACROS

Você já viu `format!`, `vec!`, `assert_eq!`. O `!` indica macro: código que **escreve código** na compilação. O projeto define as suas — procure `macro_rules!` `sym` em `emulation/core.rs`: encurta a checagem repetitiva dos 24 símbolos que o core exporta.

E os `#[derive(...)]` da aula 4? Macros de atributo — você já era fluente e nem sabia.

Último conceito do passeio: **traits** — as “interfaces” de Rust, um contrato de métodos que um tipo promete ter. Você já as usou sem ver: `#[derive(Debug)]` implementa o `trait Debug`, e o `impl Drop` da aula 8 implementa o `trait Drop`. Quando bater num `impl TraitX for TipoY`, já sabe o que é.

12

PROJETO FINAL

UMA CONTRIBUIÇÃO DE VERDADE, EM TRÊS DEGRAUS

Escolha o seu degrau e abra o editor. Todos cabem numa sessão de 30 minutos — e todos terminam com cargo test verde.

01

Degrau fácil — formatador de tamanho.

O método `human_size` (aula 3) + o teste (aula 10). Se ainda não fez, é o seu “hello world” sério: alterar, testar, ver verde.

02

Degrau médio — teste para a varredura da estante.

Explore `scan` em `crates/domain/src/library.rs` (a função que acha as ROMs na pasta) e escreva um teste seu para ela. Primeiro contato com código que você não escreveu.

03

Degrau ambicioso — um preset de TV seu.

O enum `Preset` em `crates/ntsc/src/lib.rs` define os sabores de imagem (Composite, RGB, RF...). O arquivo tem 232 linhas e é um ótimo lugar para mexer sem medo. Que tal um “Trinitron 1997”?

Os comandos do dia a dia (na raiz do projeto):

terminal

BASH

```
cargo check          # "será que compila?" (rápido)
cargo test           # roda TODOS os testes
cargo test -p xperience-domain # só os testes de um crate
cargo run -p xperience-app --bin selector # abre a estante de jogos
```

PRÓXIMOS NÍVEIS

- 1. The Rust Book** — doc.rust-lang.org/book, gratuito; depois deste curso vai parecer revisão.
- 2. Rustlings** — github.com/rust-lang/rustlings; ginástica de exercícios com o compilador corrigindo você.
- 3. Raid final:** leia `crates/platform/src/cabinet.rs` — o desenho do console, o arquivo mais visual do projeto. Se aguentar 100 linhas sem pânico, o curso cumpriu o papel.

**CONTEÚDO
100%**11 AULAS
+ PROJETO**CONQUISTAS DESBLOQUEÁVEIS**

“**Compilou de primeira**” — mentira, ninguém consegue, nem nós. “**Li o erro até o fim**” — 90% dos problemas se resolvem aqui. “**Teste verde no primeiro run**” — guarde esse dia. “**Expliquei ownership para alguém**” — aí você aprendeu de verdade.

13

COLA DE BOLSO

RASGUE E LEVE NO BOLSO (OU FAVORITE NO EDITOR)

VOCÊ QUER...	EM RUST	EM JS/CYPRESS VOCÊ DIRIA	ONDE VER NO PROJETO
criar variável	let x = 5; / let mut x = 5;	const por padrão / let	em todo lugar
texto	String(dono) / &str(empréstimo)	string só (e reza)	core.rs
lista	Vec<T> / [T; N] fixo	Array	Frame.pixels
um-de-vários	enum	union de literais	Mapper, Button
dados com nome	struct	interface/objeto	RomId, Frame
escolher por valor	match	switch (exaustivo!)	environment_cb
pode não ter	Option<T>	null/undefined	RomId.internal_name
pode falhar	Result<T,E> + ?	try/catch	RomId::from_path
transformar listas	.iter().filter().map().collect()	map/filter	score, title_at
função anônima	x x + 1	x => x + 1	em toda esteira
compartilhar	&T leitores / &mut T escritor único	passa referência e torce	Core::run / audio
testar	#[test] fn ... + assert_eq!	it(...) + expect()	fim de rom.rs
rodar	cargo build/run/test/check	npm .../cypress run	raiz do repo

OS TRÊS ERROS DO COMEÇANTE (E O REMÉDIO)

1. Lutar contra o borrow checker → leia o erro inteiro; ele sugere a correção.

2. .unwrap() por todo lado → em código de produção, prefira ? e Result.

3. Querer clonar tudo para “resolver” → o empréstimo (&) quase sempre resolve sem copiar.

DE UMA QA, PARA OUTRAS QAS

Você já pensa como um Rustacean: desconfia do runtime, confia no pipeline e quer o erro **cedo e explicado**. Rust só leva isso ao extremo — o pipeline é o compilador, e o compilador nunca fica verde por engano. Bom jogo, e bons testes.

SNES XPERIENCE · MINI CURSO DE RUST · O CÓDIGO-FONTE COMPLETO ESTÁ AO LADO: CRATES/